

ProfitView System Documentation

Version: 1.1

Document Purpose: Explain what ProfitView is, what it can do, how it is structured, and how teams can set it up, use it, and test it correctly.

Website URL: [ProfitView](#)

1) Project Overview

Project Name

- ProfitView (repository package name: accounting-saas)

What the ProfitView System Is

ProfitView is a web-based accounting and finance operations platform for organizations. It centralizes day-to-day bookkeeping and financial workflows in one workspace, including invoicing, expense capture, bills, banking, reconciliations, reporting, and team collaboration.

The system is built as a multi-organization application, so each organization has isolated records, settings, and user access.

What ProfitView Can Do

- Manage customers and vendors (contacts)
- Create and track invoices
- Record and allocate payments
- Manage credit notes
- Capture expenses and supplier bills
- Upload documents and run OCR data extraction
- Manage bank accounts and imported transactions
- Reconcile bank and accounting records

- Post journal entries and manage chart of accounts
- Generate accounting and tax-oriented reports
- Invite/manage team members with role-based access
- Send invoice emails and accept Stripe-based payments
- Run recurring automations and proactive alerts

Target Users

- Small to medium-sized businesses
- Startup operators and founders
- Finance officers/accountants/bookkeepers
- Operations admins managing teams and controls
- QA teams validating finance flows before release

Key Business Value

- Reduces manual accounting work
 - Improves financial visibility across modules
 - Creates auditable records and better compliance support
 - Supports team collaboration with permission control
-

2) System Architecture

High-Level Architecture (Simple View)

ProfitView follows a layered architecture:

1. Presentation Layer (Frontend)

- Next.js pages and React components
- Organization-specific routes: `/(app)/[orgSlug]/...`

2. Business Layer (Backend Logic)

- Server actions under `src/modules/*/actions.ts`

- Validation schemas under `src/modules/*/schemas.ts`

3. Data Layer (Database + Access Security)

- Supabase PostgreSQL
- SQL migrations under `supabase/migrations`
- Row-Level Security (RLS) policies for tenant-safe access

4. Integration Layer

- Stripe (payment checkout + webhook events)
- Resend (email delivery)
- OCR providers (OCRSpace / EdenAI)
- AI services (Anthropic/OpenAI-compatible usage paths)

Frontend / Backend / Database Overview

Frontend

- Built with Next.js 15, React 19, TypeScript, Tailwind CSS
- Organized by route groups:
 - `(auth)` for login/register/reset
 - `(orgs)` for organization selection/creation
 - `(app)` for finance modules per organization

Backend

- Logic centered in module actions (accounts, invoices, banking, etc.)
- Middleware checks auth/session behavior
- API routes are used mainly for webhooks and cron tasks

Database

- Supabase-hosted PostgreSQL schema evolved via migration files
- Covers accounting entities and automation entities
- Uses RLS to isolate org-level data

Technology Stack

- Framework: Next.js
 - Language: TypeScript
 - UI: React, Tailwind, Radix UI
 - Data/Auth: Supabase (PostgreSQL + Auth)
 - Validation: Zod
 - Email: Resend
 - Payments: Stripe
 - Testing: Vitest, Playwright, Testing Library
-

3) Installation Guide

Prerequisites

Before setup, install/provision:

- Node.js (18+ recommended)
- npm
- Supabase project (URL, anon key, service role key)
- Optional external services:
 - Stripe account (for online payments)
 - Resend account (for email)
 - OCR provider key
 - AI provider key(s)

Step-by-Step Local Setup

1. Clone repository `git clone` (private repo for now)
`cd ProfitView`
2. Install dependencies `npm install`
3. Create local environment file `cp .env.example .env.local`
4. Configure ``.env.local`` values (see list below)
5. Apply Supabase migrations

- Use your standard Supabase migration workflow to apply files under `supabase/migrations`.
- 6. Run development server `npm run dev`
- 7. Open application
 - Visit: `http://localhost:3000`

Environment Variables Configuration

Required (minimum)

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `SUPABASE_SERVICE_ROLE_KEY`
- `NEXT_PUBLIC_SITE_URL`

Optional (feature-dependent)

- `CRON_SECRET` – protects cron endpoints
- `RESEND_API_KEY`, `FROM_EMAIL` – email features
- `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET` – Stripe features
- `OCR_PROVIDER`, `OCR_SPACE_API_KEY`, `EDENAI_API_KEY`, `OCR_API_URL` – OCR features
- `ANTHROPIC_API_KEY`, `GITHUB_TOKEN` – AI/agent features

Local Run Commands

- Start dev mode: `npm run dev`
 - Build production bundle: `npm run build`
 - Start production build: `npm run start`
 - Run tests: `npm run test`
 - Run e2e tests: `npm run test:e2e`
-

4) Usage Guide (Step-by-Step)

This section is intended for end users, testers, and demo presenters.

A. First-Time User Flow

1. Register a new account (or log in)
2. Create/select organization
3. Access organization dashboard
4. Set baseline settings:
 - ☐ Accounting periods
 - ☐ Tax rates
 - ☐ Team members/roles

B. Core Operational Flow (Recommended Test Journey)

Use this sequence to properly evaluate the system end-to-end:

1. Create master data
 - ☐ Add chart accounts (if needed)
 - ☐ Add contacts (customer + supplier)
2. Test sales flow
 - ☐ Create invoice
 - ☐ Send/preview invoice
 - ☐ Record payment or simulate Stripe checkout
3. Test purchase/expense flow
 - ☐ Add expense/bill
 - ☐ Attach receipt/invoice file
 - ☐ Trigger OCR extraction (if configured)
4. Test banking flow
 - ☐ Create bank account
 - ☐ Import/add transactions
 - ☐ Apply rules/categorization
 - ☐ Reconcile against accounting entries

5. Test reporting flow

- Open Trial Balance
- Open General Ledger
- Open Aging reports
- Verify Balance Sheet/VAT outputs

6. Test controls/audit flow

- Create journal entry
- Lock period scenario
- Check audit log entries

C. Module-by-Module Explanation

- Dashboard: Overview of key financial indicators and trends
 - Contacts: Customer/vendor profile and history
 - Invoices/Credits: Sales documents and settlement lifecycle
 - Expenses/Bills: Outgoing obligations and expense tracking
 - Payments: Allocation and payment status management
 - Banking: Account registry, transactions, reconciliation worksheets
 - Journal: Manual accounting adjustments
 - Reports: Financial statements and compliance reports
 - Settings: Team, taxes, periods, and organization preferences
-

5) API Documentation (Applicable Routes)

ProfitView primarily uses server actions internally. The exposed route handlers are mostly system endpoints.

Endpoint List

1. POST /api/cron/proactive-alerts
2. POST /api/cron/recurring
3. POST /api/cron/agent-alerts

4. POST /api/webhooks/stripe

Request/Response Format (General)

1) Cron Endpoints

- Method: POST
- Auth: Authorization: Bearer <CRON_SECRET> (if secret configured)
- Purpose: Trigger automation jobs
- Response: JSON status (success/failure + process details)

Example:

```
curl -X POST http://localhost:3000/api/cron/recurring \
-H "Authorization: Bearer YOUR_CRON_SECRET"
```

2) Stripe Webhook

- Method: POST
- Auth model: Stripe signature verification
- Purpose: Receive payment events and sync invoice/payment state
- Validation key: STRIPE_WEBHOOK_SECRET

Local forwarding example:

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

6) Database Structure

Main Data Domains (Conceptual)

- Organization/workspace and membership
- Auth and role mapping
- Settings, periods, tax rates
- Accounts (chart of accounts)
- Contacts
- Invoices and invoice payments
- Credits and applications
- Expenses
- Bills and bill payments
- Journal entries and lines
- Banking: accounts, transactions, reconciliation, rules
- Attachments and OCR extraction records
- Reports support entities (e.g., VAT returns)
- Invitations/team collaboration
- Audit logs
- Recurring schedules/expenses
- Agent sessions, approvals, memory, processing state

Relationship Summary

- One organization contains many finance entities
- Contacts link to invoices/bills depending on role
- Invoices may have many payments and credits
- Journals contain multiple debit/credit lines
- Bank records connect to reconciliation outcomes
- Audit logs track actions across modules for traceability

7) Code Structure

Repository Layout

- `src/app/` – Route-based UI and API route handlers

- `src/modules/` – Domain logic (actions, schemas, types)
- `src/lib/` – Shared utilities and integrations
- `supabase/migrations/` – Database migration scripts
- `src/test/` – Integration test setup/helpers

Major Components Explained

- Route Layer: User-facing pages, per-module screens
- Module Layer: Business operations and server actions
- Validation Layer: Input validation and schema constraints
- Data Client Layer: Supabase clients (client/server/service/middleware)
- External Adapter Layer: Stripe/Resend/OCR/AI wrappers

Engineering Patterns

- Feature/domain-based organization
 - Tenant-aware access with org context (orgSlug)
 - Separation of UI and domain operations
 - Explicit schema validation before writes
-

8) Features Breakdown (How It Works Internally)

8.1 Authentication & Access Control

- Handles login/registration/reset
- Uses Supabase auth sessions
- Middleware and RBAC utilities gate protected functionality

8.2 Invoicing & Collections

- Invoice lifecycle: draft → issued → paid/partially paid/voided
- Payment recording updates invoice balances

- Optional Stripe checkout creates online payment sessions
- Email pathways support invoice notifications/reminders

8.3 Credits

- Create credit notes and apply them to invoices
- Supports outstanding balance reduction logic

8.4 Expense & Bills Management

- Records outgoing financial obligations
- Supports bill payment tracking and status updates
- File attachments provide source documentation

8.5 OCR and Document Intelligence

- OCR provider is configurable
- Extracted values support faster bill entry and validation

8.6 Banking & Reconciliation

- Track bank accounts and imported lines
- Rules assist categorization/automation
- Reconciliation tools align bank vs ledger records

8.7 Accounting Core

- Account master data + journal entries support full bookkeeping
- Period controls enforce closing/locking discipline

8.8 Reporting

- Trial balance, general ledger, aging, balance sheet
- VAT-related reports and return workflows

8.9 Team & Auditability

- Invite members and manage access
- Audit logs provide transparency and traceability

8.10 Automation & Agent

- Scheduled cron tasks trigger recurring operations
 - Agent subsystem includes approvals, memory, and proactive workflows
-

9) Troubleshooting

Common Issues and Fixes

1. Application does not start

- Check required env variables in `.env.local`
- Re-run `npm install`

2. Supabase auth/data errors

- Verify URL/keys
- Confirm migrations are applied
- Validate RLS and org membership records

3. Unauthorized cron execution

- Ensure request includes correct bearer token from `CRON_SECRET`

4. Stripe webhook verification fails

- Confirm you are using the matching signing secret for your current listener/environment

5. Email not sending

- Check `RESEND_API_KEY` and sender email/domain setup

6. OCR returns empty/failed extraction

- Confirm selected OCR provider and correct API key
- Test with clear sample document quality

7. Unexpected permission behavior

- Verify user role assignment and organization context
-

10) Future Improvements

Product/Feature Enhancements

- Guided onboarding wizard for first-time organizations
- Rich dashboard KPIs and customizable widgets
- Approval workflows for bills/payments/journals
- Native multi-currency reporting and FX revaluation automation
- Expanded tax localizations beyond current setup

Technical/Scalability Enhancements

- OpenAPI-style documentation for endpoint/server-action contracts
- Dedicated background worker queues for heavy async jobs
- Stronger observability (metrics, traces, alerting)
- More resilient disaster recovery and backup verification
- Deeper automated test coverage of end-to-end accounting scenarios

UX/Adoption Enhancements

- In-app contextual help and walkthroughs
 - Report export templates for finance stakeholders
 - Better audit timeline visualizations
-

Appendix A: Quick Start for Demo/Test Teams

1. Configure environment variables
2. Start app (npm run dev)

3. Register and create organization
4. Create one customer and one supplier
5. Create one invoice and mark payment
6. Create one expense/bill and attach document
7. Create one bank account and add transaction
8. Reconcile one transaction
9. Open Trial Balance and General Ledger
10. Review audit log entries

This 10-step path validates the most important system capabilities in a single pass.

Appendix B: Command Checklist

- `npm install`
- `npm run dev`
- `npm run test`
- `npm run test:e2e`
- `npm run build`
- `npm run start`